Chair of Methods for Model-based Development in Computational Engineering
RWTH AACHEN UNIVERSITY

The present work was submitted to the Chair of Computational Science and Engineering.

presented by:

Enes Çağlar Korkmazgöz

464051

Simulation Sciences Seminar Project Report

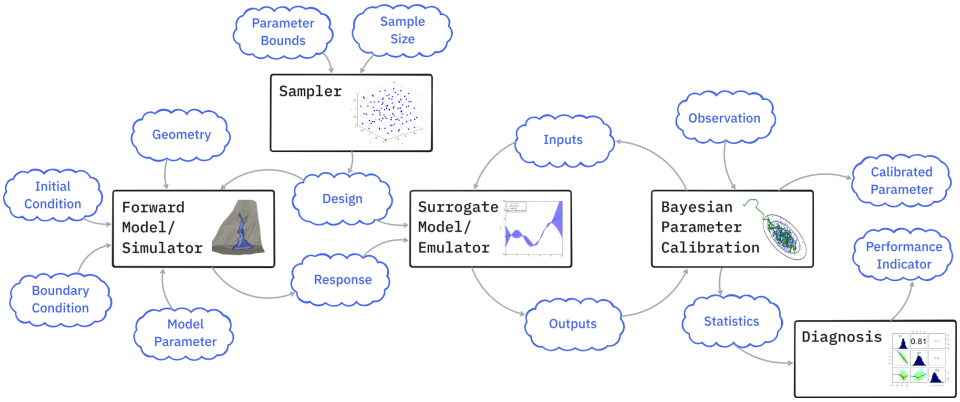
Benchmarking of Sampling Methods for Probabilistic Calibration in Gravity-Driven Mass Flow Modeling

Supervisors :

Alan Correa

Mithlesh Kumar

Aachen, *15.08.2026*



Benchmarking of Sampling Methods for Probabilistic Calibration in Gravity-Driven Mass Flow Modeling

Enes Çağlar Korkmazgöz

Abstract

Bayesian calibration of gravity-driven mass flow models requires repeated exploration of a posterior distribution and may depend strongly on the selected sampling method. This report presents `mcmc-bench`, a modular framework that integrates Random-Walk Metropolis-Hastings, emcee, dynesty, PyMC Slice, and PyMC Sequential Monte Carlo through Nextflow and UM-Bridge. All methods calibrate the Voellmy friction parameters μ and ξ using the same RobustGaSP surrogate, observations, priors, and likelihood. Across the three executions, all samplers produced similar posterior means and standard deviations. Under the selected sampler configurations, RWMH had the shortest runtime and required the fewest surrogate-model evaluations, while RWMH and dynesty produced similar results according to the normalized ESS metrics. However, the available metrics do not support a clear overall ranking of the samplers.

Declaration of Own Work

I hereby declare that this thesis has been written independently and that no sources other than those cited have been used. All passages taken verbatim or paraphrased from published or unpublished works have been clearly marked as such. This work has not been submitted, in whole or in part, for any other academic qualification.

Declaration of AI Tool Usage

The following AI tools were used during the preparation of this thesis:

- **ChatGPT 5.6 Sol** — This assistant was used for grammar checking of the report and debugging of the source code throughout the project. The author takes full responsibility for the accuracy, interpretation, and final content of this report.
- **Google Scholar LABS** — This tool was used to assist in the literature review process by providing relevant academic papers and references.

All AI-assisted content has been reviewed and verified by the author. No AI tool was used to generate results, draw conclusions, or replace independent scientific reasoning.

Aachen, *31.07.2026*

Enes Çağlar Korkmazgöz

Contents

List of Figures

List of Tables

1 Introduction

Computational models are widely used to represent and predict the behaviour of complex systems, but their predictions often depend on input parameters that are not accurately known or may be completely unknown for the application under study. Model calibration uses observations of the physical system to learn about these parameters. The relationship between observations and model parameters is important not only for improving model predictions, but also for quantifying the uncertainty that remains in the inferred parameters and subsequent predictions.

Bayesian inference provides a framework for *learning from experience*, and we can utilize it with our two sources of information. First, existing knowledge or assumptions about the quantities that are unknown to us can be represented by the prior distribution, while observations can be used in the likelihood calculations. As a result, we obtain the posterior distribution, which expresses what has been learned after observing the data. Bayesian methods are not restricted to a particular application area but are widely used to estimate uncertain parameters and predictions across a broad range of scientific areas [23].

These parameters of interest can have different interpretations. For example, physical parameters have meaning within the scientific description of a system, and it can be difficult to measure them directly from the physical system. Moreover, tuning parameters cannot be directly measured from the system either, not because of the difficulty of measuring them, but because they are not physically available parameters. Instead, they are generally simplified parameters that represent processes which the model does not explicitly resolve, and they are adjusted according to the observations to improve the model fit [4]. Therefore, once we determine the type of parameter to be solved, we may answer the following question: are we learning about a physical parameter and hence gaining information about the system, or are we looking for a tuning parameter that remains specific to the model? In this study, the Voellmy model parameters are considered, where the μ and ξ enter the resistance law as dry-Coulomb and turbulent friction coefficients, respectively. However, they are effective, conceptual parameters rather than directly measurable

material constants, and their values are commonly inferred by analysing the scenarios from the field [14, 31].

Bayesian calibration typically requires repeated evaluations of the forward model, which might be expensive. That is why computationally cheaper models, namely emulators, can be used to approximate the input-output relationship of the forward model.

The source code, workflow definitions, and configuration files of this study are publicly available in the `mcmc-bench` GitHub repository.

1.1 Motivation

Gravity-driven mass flow models simulate the runout of avalanches, debris flows and related flow-like landslides, providing estimates of quantities such as flow depth, velocity, and impact area, among others (see Figure 1.1 as an example). These are employed for quantitative risk assessments and can inform mitigation planning and decision support [31]. Their practical value, however, depends on the reliability of the simulated event behaviour. Because the effective resistance parameters cannot be measured directly, they must be calibrated against observations, and the uncertainty associated with this calibration must be considered when interpreting model predictions [14, 31].

A deterministic calibration can be formulated as an optimization problem that returns a parameter vector minimizing the discrepancy between model outputs and observations. However, observations contain measurement error. If these sources of error are not represented explicitly, the fitted parameters may include noise leading to biased conclusions [4, 16]. Calibration may also suffer from identifiability problems: different parameter combinations can explain the observations similarly well [2]. Consequently, a single best parameter value does not account for the result. The relevant question is instead: *which combinations of parameter values are plausible, and how can we determine their uncertainty?*

Bayesian calibration addresses this problem by representing uncertainty about model parameters through a posterior distribution rather than a single estimate. Reliable predictive uncertainty quantification requires consideration of model discrepancy and surrogate uncertainty [4, 31]. Such a study is not evaluated in this benchmark; here, the posterior serves as the common target distribution for all of the samplers.

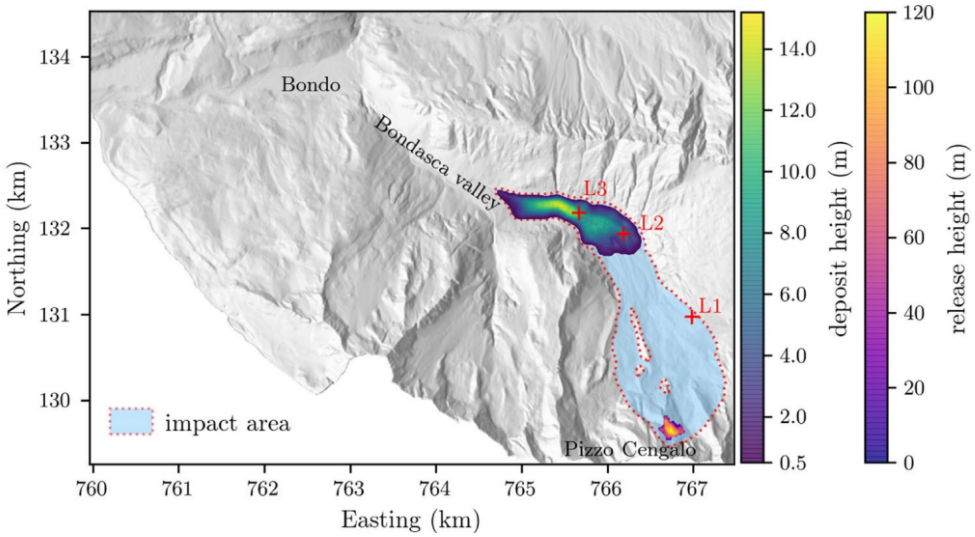


Figure 1.1: Topography and release mass distribution of the 2017 Bondo landslide, together with the simulated impact area and deposit distribution for Voellmy parameters $\mu = 0.23$ and $\xi = 1000\,\mathrm{m\,s^{-2}}$. Reproduced without modification from Zhao and Kowalski [32, Fig. 2] under the CC BY 4.0 license.

When posterior quantities cannot be computed analytically, sampling algorithms are used to explore the target distribution. Different sampling algorithms approach posterior calculations differently. Therefore, their relative accuracy and efficiency can depend on various factors, including implementation details and the shape of the target distribution [1].

These sampling algorithms are implemented by different individuals and groups and distributed as packages with their own configuration formats, dependencies, and outputs. Comparing them on a calibration problem therefore requires the problem to be adapted to each sampler. This makes it difficult to determine whether observed differences arise from the sampling methods or from their implementations. Systematic comparisons have been carried out in other application areas. For example, Albert et al. [1] compare Bayesian sampling algorithms for high-dimensional particle physics and cosmology applications, while gravity-driven mass flow studies have focused on individual calibration or uncertainty-quantification approaches rather than comparing multiple samplers [14, 31, 32].

This leads to two gaps. Firstly, there is no common framework for this application that keeps the surrogate model, observations, prior, and likelihood fixed and changes only the sampler. So, samplers cannot be compared fairly without a common framework. Secondly, it is not clear which criteria such a comparison should

use. The generic convergence diagnostic criteria were developed mostly for Markov chains, so they cannot be read the same way for samplers that produce interacting walkers, particle populations, or weighted samples. As a result, the comparison needs both a common set of criteria and rules for interpreting them per method.

1.2 Research Questions

Our research questions are as follows:

RQ1 How can different sampling methods be integrated into a modular, extensible, and reusable benchmarking framework for Bayesian calibration of a surrogate-based gravity-driven mass flow model?

RQ2 Which accuracy, convergence, and computational-efficiency criteria enable a meaningful comparison of the integrated sampling methods, and how do the samplers perform according to these criteria for the chosen Voellmy calibration case study?

1.3 Outline

The following chapters of this report are organized as follows. In Chapter 2, we introduce inverse problems, Bayesian calibration, surrogate modeling, and the selected sampling methods. In Chapter 3, we continue with the common benchmarking framework and the integration of the five samplers. In Chapter 4, we present the experimental setup, comparison criteria, results, and discussion. Finally, in Chapter 5, we summarize the findings and outline future work.

2 Background and Related Work

2.1 The Inverse Problem

Forward models map a set of input parameters to observable outputs. An inverse problem begins with the observations and searches for input or model parameters that are consistent with them. Model calibration performs this inference by comparing simulator outputs with observed data. The inverse relationship is generally not unique. Several parameter combinations may explain the observations equally well, especially in the presence of measurement uncertainty and discrepancies between the simulator and reality [2, 16]. Figure 2.1 illustrates the relationship between the forward and inverse problems.

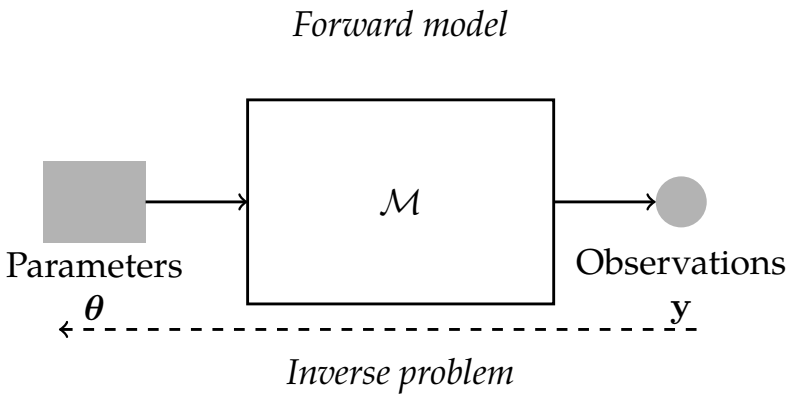


Figure 2.1: Relationship between the forward and inverse problems. The forward model maps parameters $\boldsymbol{\theta}$ to predicted observables, whereas the inverse problem uses observations $\mathbf{y}$ to obtain plausible parameter values. Adapted from the conceptual illustration by Moura Neto and da Silva Neto [17].

In inverse-problem theory, a problem is called ill-posed if a solution does not exist, is not unique, or changes strongly under small changes in the data [17, Chap. 3]. Calibration problems often face the latter two issues: because of measurement noise, limited data, and model error, several different parameter combinations can fit the

observations equally. Two common solutions are regularization and probabilistic methods, which add extra information about the unknown parameters [3, 27]. This study follows the probabilistic approach. In gravity-driven mass flow modeling, Bayesian calibration is often used, which estimates effective friction parameters from observed events and also quantifies the remaining uncertainty [14, 32].

2.2 The Bayesian Approach to the Inverse Problem

Bayesian calibration treats the unknown model parameters as uncertain quantities and uses Bayes' theorem to update prior information with observed data. Let θ denote the model parameters and y the observations. The prior distribution $p(\theta)$ represents information about the parameters before considering y, while the likelihood $p(y \mid \theta)$ describes how compatible the observations are with a given parameter value. Bayes' theorem combines these components into the posterior distribution:

$$p(\theta \mid y) = \frac{p(y \mid \theta)\, p(\theta)}{p(y)}, \tag{2.1}$$

where $p(y)$ is the model evidence, which does not depend on θ and normalizes the posterior distribution.

The evidence can be written as $p(y) = \int p(y \mid \theta) p(\theta)\, \mathrm{d}\theta$, but this integral is generally not available to us. Because $p(y)$ is constant with respect to θ, many MCMC methods can explore the posterior using only its unnormalized density [1, 16]:

$$p(\theta \mid y) \propto p(y \mid \theta)\, p(\theta). \tag{2.2}$$

In computation, the unnormalized posterior is commonly evaluated on the logarithmic scale. This converts the product into a sum and reduces numerical errors (in floats) when the probability densities are very small:

$$\log \tilde{p}(\theta \mid y) = \log p(y \mid \theta) + \log p(\theta), \tag{2.3}$$

where $\tilde{p}(\theta \mid y)$ denotes the unnormalized posterior density. While many MCMC methods require only the unnormalized posterior, nested sampling also estimates the model evidence by rewriting it as a one-dimensional integral of the likelihood over prior mass [25]. In `dynesty`, this approach is implemented by supplying the likelihood and prior-transformation functions separately [26].

The likelihood connects the observations to the output of the forward model. The calibration framework uses an additive Gaussian residual model:

$$y_i = f_i(\theta) + \varepsilon_i, \qquad \varepsilon_i \overset{\text{iid}}{\sim} \mathcal{N}(0, \sigma^2), \tag{2.4}$$

where $f_i(\theta)$ denotes the model prediction corresponding to observation y_i, and σ is the residual standard deviation. Under this assumption, the log-likelihood for n observations is

$$\log p(y \mid \theta, \sigma) = -\frac{n}{2}\log(2\pi\sigma^2) - \frac{1}{2\sigma^2}\sum_{i=1}^{n}(y_i - f_i(\theta))^2. \tag{2.5}$$

For the general formulation, $f_i(\theta)$ denotes the full forward-model response. In the benchmark implementation, it is replaced by the surrogate prediction $\hat{f}_i(\theta)$.

The residual term may combine the model error and measurement error. Consequently, independent Gaussian residuals are assumptions rather than a property guaranteed by the observations [4, 14, 16].

For a Bayesian calibration, each likelihood evaluation generally requires a separate model simulation. For full computational models, this is generally expensive. Therefore, one may utilize a surrogate model that approximates the input-output relationship of the original forward model using a selected set of precomputed runs from an initial training data set [12, 31]. As a result, the cost of repeated model evaluations is reduced, but the original model response is now replaced by the approximations produced by the surrogate model. Therefore, its accuracy and uncertainty must be considered when interpreting the posterior obtained from these results [12, 31, 32].

In this study, instead of using the full gravity-driven mass flow simulator, the likelihood is calculated using the RobustGaSP surrogate. Even though a surrogate evaluation is cheaper than a full model run, it is still performed repeatedly, and different sampling methods may require different numbers of evaluations to reach the desired results.

2.3 The Bayesian Calibration Pipeline

So far, we have seen that Bayesian calibration assigns prior distributions to the unknown parameters, the forward model or surrogate model maps these parameters to predicted observables, and the likelihood compares these predictions with the

observed data. These components should be kept separate, since changing the surrogate, prior, likelihood, or observations may lead to a change in the target posterior, whereas changing the sampler only changes the numerical approach used to approximate the target distribution.

Figure 2.2 provides a broader view of this process, including the construction of a surrogate model from the forward model, as well as the calibration and diagnostic stages.

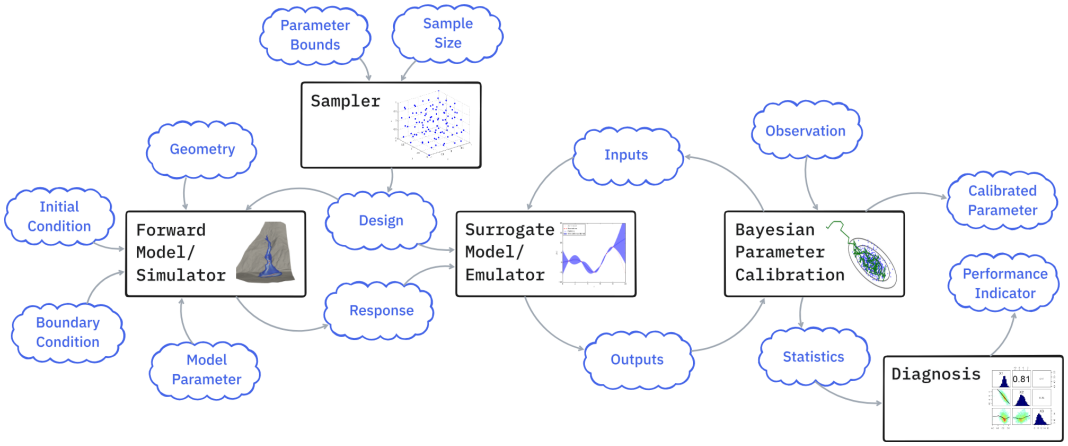


Figure 2.2: General workflow of Bayesian calibration. Illustration courtesy of Alan Correa.

The `mcmc-bench` framework implements this concept by separating model evaluation from sampler execution. Here, the RobustGaSP surrogate is used through UM-Bridge, so sampler implementations interact with it through a common, language-agnostic interface rather than depending directly on the surrogate's internal implementation [12, 24, 31]. Nextflow coordinates the sampler-specific software environments and the diagnostic and reporting steps within a reproducible computational workflow [7].

2.4 Sampling Methods for Bayesian Inference

Modern Bayesian computation includes gradient-based methods such as HMC and NUTS, ensemble and slice-based approaches, population methods such as Sequential Monte Carlo, nested-sampling algorithms, and more recent methods that use normalizing flows to construct proposals or transform the sampling space. Representative implementations use several programming languages and software packages, as summarized in Table 2.1.

Table 2.1: Selected state-of-the-art sampling algorithms.

Algorithm	Package(s)	Language(s)	Benefit(s)
HMC/NUTS [8, 15]	PyMC [21]	Python	Gradient-informed exploration
Affine-invariant ensemble [10, 11]	emcee	Python	Robustness to linear rescaling and correlation
Slice sampling [18]	PyMC [22]	Python	Adaptive interval construction
SMC [6]	PyMC [20]	Python	Population-based exploration
Nested sampling [5, 25, 26]	dynesty, UltraNest	Python	Posterior and evidence estimation
Flow-based nested sampling [29]	nessai	Python	Learned constrained-prior proposals
Flow-enhanced MCMC [30]	flowMC	Python	Learned global proposals for complex posteriors

Given the time constraints of the present work, the initial benchmark integrations focus on Python-based packages. This is an implementation-scope decision rather than a limitation of the underlying framework.

This study includes five gradient-free sampling methods. Random-Walk Metropolis-Hastings uses local random-walk proposals with an accept/reject decision [13]. Affine-invariant ensemble sampling uses interacting walkers and reduces sensitivity to linear transformations of the parameter space [10, 11]. Slice sampling constructs and samples from a region under the probability density around the current state [18]. Sequential Monte Carlo evolves a population of weighted particles through a sequence of intermediate distributions [6]. Nested sampling explores constrained likelihood regions and additionally estimates the model evidence [25, 26]. These methods have different tuning requirements and computational costs [1].

In the benchmark, these methods are represented by the custom `rwmcmc` implementation, `emcee`, the Slice and SMC implementations in PyMC, and `dynesty`, respectively. Their integration into the common framework is described in Chapter 3.

3 Benchmarking Sampling Methods

This chapter presents the implementation of `mcmc-bench`, a common benchmarking framework integrating Random-Walk Metropolis-Hastings, emcee, dynesty, PyMC Slice, and PyMC Sequential Monte Carlo. In this study, the surrogate model, observations, prior distributions, likelihood, and quantification are fixed, and only the sampler integration is changed. As a result, all samplers produce outputs in a common format so that their accuracy, convergence behavior, and sampling runtime can be compared consistently. The implementation details are presented below, followed by the integration details for each sampler.

3.1 Implementation Details

The runs are executed using a single YAML configuration file in which the surrogate model, observations, calibrated parameters, prior distributions, likelihood, and sampler-specific settings are defined. In this benchmark, the Voellmy friction parameters μ and ξ are calibrated against the observed impact area in ground_truth.csv using the uniform priors $\mu \sim \mathcal{U}(0.02, 0.50)$ and $\xi \sim \mathcal{U}(400, 4000)$. For the likelihood calculations, the residual is assumed to follow a Gaussian distribution with mean zero and a fixed standard deviation of $\sigma = 0.05$. This standard deviation can be changed, and additional noise calibration can be performed, but this is outside the scope of this study. To sum up, in this calibration problem, we only modify the sampler and its configuration.

The RobustGaSP surrogate is exposed as an HTTP model using UM-Bridge. The samplers connect to the same model server with different server instances through this interface and send μ and ξ values to it. Normally, the surrogate can provide full predictive statistics, but the model server returns only the predictive mean value, which is then used in the likelihood calculation within the sampling algorithm.

```
output = self.model.predict(x)
# ScalarGaSP: (n, 4) -> [mean, lower, upper, std]
return [output[:, 0].tolist()]
```

```
# only column 0 = mean values are returned to the sampler
```

The UM-Bridge server is configured with a single worker. Also, in this study, we do not expose gradients through UM-Bridge. In addition, it has not been established whether RobustGaSP can provide the required gradients even if the server allowed us to access them. Therefore, gradient-based samplers are outside the scope of this report.

The whole process is orchestrated by a Nextflow workflow. It starts by collecting the surrogate and observation data and then creates the shared communication directory used by the UM-Bridge server. The workflow then starts the sampler process. The sampler waits until the server signals that it is ready, then sends proposed parameter values to the server during the sampling operation. After sampling is complete, the sampler writes its outputs and the server is stopped. The diagnostic and reporting processes are then executed.

The same diagnostics and report-generation steps are applied to every sampler because all of the implementations follow a common output workflow. Each sampler produces a posterior trace, log-probability values if available, the measured sampling runtime, the number of surrogate-model evaluations, a corner plot, and an ArviZ InferenceData (iData) file. The iData representation is then used to generate trace, posterior, and autocorrelation plots, together with effective sample size (ESS) and R-hat diagnostic statistics, if applicable. These files, the run configuration, and the automatically generated report are collected in the session-specific output directory.

3.2 Sampler Integration

3.2.1 Random-Walk Metropolis-Hastings

RWMH uses the custom `rwmcmc` package and evaluates the common log-posterior. The chains are initialized independently and executed serially, each with a separate random-number generator seed. The proposal step sizes can be configured separately for the parameters of interest, so different numerical scales of the parameters can be accommodated. After sampling, the configured burn-in period is removed, the chains are reshaped into the common output format, and an ArviZ iData object is written explicitly for the shared diagnostics.

3.2.2 emcee

The emcee integration passes the same log-posterior function to an `EnsembleSampler`. The walkers' positions are initialized independently. The number of walkers, sampling steps, and burn-in period can be modified in the configuration file. Normally, this package supports serial execution as well as thread- and process-level pools; however, the current benchmark uses only serial model evaluations. Following sampling, the configured burn-in period is removed from the saved posterior arrays. Similarly, after the output is converted to an iData object, the burn-in period is removed from that object as well.

3.2.3 dynesty

The `dynesty` package implements nested sampling rather than an MCMC method. Therefore, the likelihood and prior are supplied separately. A transform map is used to map points from the unit hypercube to the configured parameter distributions using their inverse cumulative distribution functions. The nested sampler is controlled by the number of live points, the evidence-based stopping criterion, and the number of worker processes. By nature, nested sampling produces weighted samples, which are converted into an equally weighted trace in this implementation for compatibility with the common diagnostic tools. The estimated log-evidence and its uncertainty are additionally saved in the sampler output.

3.2.4 PyMC Slice and Sequential Monte Carlo

Listing 3.1: PyTensor adapter used to expose the UM-Bridge likelihood to PyMC.

```
class UMBridgeLogLikelihood(Op):
    itypes = [pt.dvector]
    otypes = [pt.dscalar]

    def __init__(self, log_likelihood):
        self.log_likelihood = log_likelihood

    def perform(self, node, inputs, outputs):
        (parameters,) = inputs
        value = self.log_likelihood.eval(parameters)
        outputs[0][0] = np.asarray(value, dtype=np.float64)
```

Both PyMC integrations follow PyMC's documented black-box likelihood pattern by passing the numerical UM-Bridge likelihood to PyTensor through the operation shown in Listing 3.1 [19]. The operation accepts a "parameter" vector, evaluates the likelihood, and returns a scalar log-likelihood. Both methods return an iData object directly, from which the posterior samples are reshaped into the common output format.

4 Evaluation

In this chapter, we evaluate the five sampler integrations using the same calibration problem. In Chapter 3, we addressed RQ1 by describing how the samplers were integrated into the modular benchmarking pipeline. This chapter addresses RQ2 by showing posterior agreement, convergence behavior, and computational efficiency within a common benchmarking framework. Again, the surrogate model, observations, priors, and likelihood are held fixed; hence, the comparison focuses on differences arising from the sampling methods and their configured settings.

4.1 Experimental Setup

All methods were used to calibrate the Voellmy parameters μ and ξ against the observed impact-area value $y = 0.56139776$. The problem is already described in Section 3.1. The surrogate-model server used one worker. The sampler integrations were also restricted to one worker, process, or core, depending on the sampler. Table 4.1 summarizes the sampler-specific settings used in all three executions.

None of the samplers were forced to produce the same number of stored samples because their computational controls are not equivalent. RWMH, emcee, and PyMC Slice use fixed chain or walker lengths, while PyMC SMC uses a particle-population size, and dynesty, on the other hand, stops according to its evidence-based criterion. As a result, obtaining the same number of stored samples would not imply the same computational load or cost. Instead, the statistical target and sampler-specific configurations were held fixed, while we recorded the computational cost using the number of surrogate-model calls and the sampling time for each sampler.

Each sampler was evaluated for three separate executions to assess variability and hence reduce dependence on a single result. Each execution used a separate random-number initialization. The statistical problem and the sampler settings that determine the computational budgets, proposal mechanisms, and stopping criteria were kept unchanged across the three executions.

Firstly, the posterior summaries and computational quantities were calculated separately for each run. Unless otherwise stated, values reported in the following sections as *mean* $\pm$ *SD* are the arithmetic mean of the three execution-level values and their sample standard deviation across the executions. In posterior standard deviation columns, the first value is the average of the three execution posterior standard deviations, while the value after $\pm$ describes how this value varied between each execution. Sampling time covers only the sampler execution and excludes environment creation, diagnostic generation, and report generation.

The benchmark was executed on Ubuntu under WSL2 using an Intel Core i7-13700H processor with 20 logical processors and 15 GiB of memory available to WSL. The workflow used Nextflow 25.10.4. The sampler environments used Python 3.12.13 and ArviZ 0.20.0, together with rwmcmc 0.1.1, emcee 3.1.6, PyMC 5.26.1 with PyTensor 2.35.1, and dynesty 3.1.0. The surrogate environment used Python 3.14.6, R 4.4.3, psimpy 1.0.0, UM-Bridge 1.2.9, and RobustGaSP 0.6.4.

Table 4.1: Sampler configurations used in all three benchmark executions.

Sampler	Configuration
RWMH	4 chains; 5000 steps per chain; 1000 burn-in steps; proposal scales $(0.15, 1200)$
emcee	8 walkers; 5000 steps per walker; 1000 burn-in steps; serial execution
PyMC Slice	4 chains; 1000 tuning steps and 4000 retained draws per chain; one core
PyMC SMC	4 chains; 2000 draws per chain; threshold 0.5; correlation threshold 0.01; one core
dynesty	3000 live points; stopping tolerance $\Delta \log Z = 0.1$; one process

4.2 Metrics

The comparison we consider has the following three aspects. They are posterior agreement, convergence, and computational efficiency.

4.2.1 Posterior Agreement

An analytical posterior is not available to us as a reference for this study. So, posterior agreement can be evaluated by comparing the posterior means, standard devi-

ations, and joint distributions generated by the samplers. By doing so, we measure the consistency of the numerical approximations across the samplers, but unfortunately, this does not provide an absolute measure of posterior accuracy.

In this case study, this agreement is determined by the posterior means and standard deviations of the Voellmy friction coefficients. Comparisons of corner and histogram plots may also help to decide whether agreement is achieved. One may expect similar numerical results and distribution shapes for the posterior distributions in the case of agreement. If, however, there are clear differences between the outputs of the samplers, this may indicate disagreement, as well as insufficient exploration or sensitivity to the sampler settings, which may need to be further tuned.

4.2.2 Convergence

For the convergence study, there are multiple aspects to assess. For Markov chain outputs, convergence is assessed using the R-hat, ESS-bulk, and ESS-tail diagnostics. R-hat compares the variation between and within different chains within an execution, and a value below 1.01 indicates that no clear convergence problem exists. ESS-bulk and ESS-tail describe the amount of information in the central region and in the tails of the posterior distribution, respectively [1, 28].

For the samplers in this study, the diagnostics require different interpretations because their outputs have different structures. For emcee, the walkers communicate during sampling, and therefore they are not treated as independent chains for the R-hat diagnostic [10, 11]. Dynesty, on the other hand, produces weighted nested-sampling points rather than Markov chains. As a result, R-hat is not applicable to this output, yet the dynesty implementation has an evidence-based stopping criterion [26]. Similarly, PyMC SMC does not produce Markov chains but instead produces independent particle populations, for which the R-hat value is interpreted solely as an indicator of agreement between repeated populations, not chains [6].

4.2.3 Efficiency

For the efficiency assessment, computational costs are measured using the total sampling time and the number of successful surrogate calls. These surrogate calls provide a hardware-independent indication of the computational demand of the sampler. They are particularly relevant when the run requires repeated model evaluations of an expensive forward model or its surrogate [31]. Moreover, sampling time captures sampler and software overhead in the experimental environment. To

Table 4.2: Posterior summary statistics across the three executions, reported as mean ± sample SD across executions. In the posterior-SD columns, the first value is the average posterior SD from the three executions.

	μ		ξ	
Sampler	Mean	Posterior SD	Mean	Posterior SD
RWMH	0.18509 ± 0.00125	0.08369 ± 0.00142	2334.5 ± 35.2	954.8 ± 6.0
emcee	0.19103 ± 0.00037	0.08314 ± 0.00119	2365.9 ± 14.0	944.4 ± 9.5
PyMC Slice	0.18595 ± 0.00042	0.08379 ± 0.00113	2336.6 ± 6.8	956.0 ± 3.6
PyMC SMC	0.18645 ± 0.00032	0.08343 ± 0.00059	2349.4 ± 6.6	951.0 ± 3.8
dynesty	0.18546 ± 0.00110	0.08358 ± 0.00046	2336.2 ± 4.2	952.5 ± 4.9

relate computational costs to posterior information, the benchmark uses the minimum bulk ESS across the calibrated parameters for RWMH, emcee, PyMC Slice, and PyMC SMC, and the Kish ESS for dynesty [9],

$$\mathrm{ESS}_{\mathrm{min}} = \min\left(\mathrm{ESS}_{\mathrm{bulk},\mu}, \mathrm{ESS}_{\mathrm{bulk},\xi}\right),$$

and reports

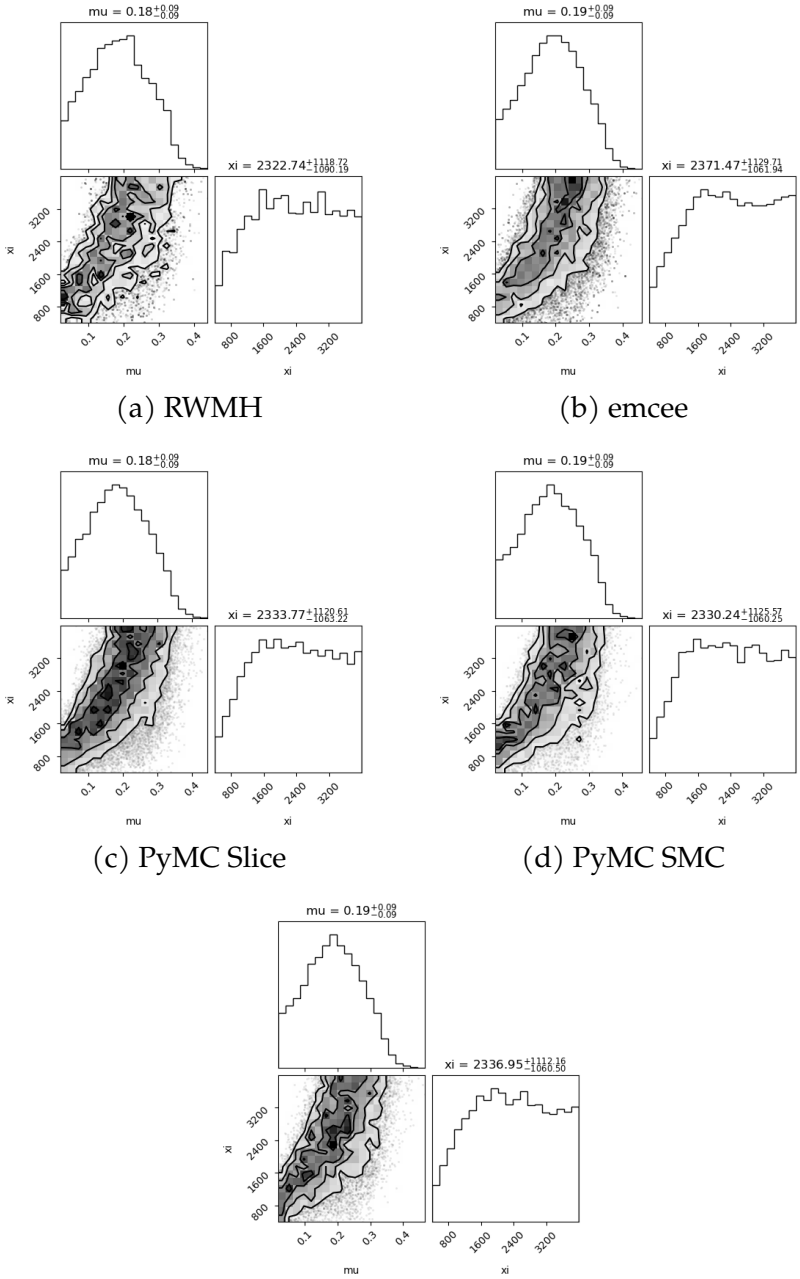
$$\text{ESS per evaluation} = \frac{\mathrm{ESS}_{\mathrm{min}}}{N_{\mathrm{eval}}}, \qquad \text{ESS per second} = \frac{\mathrm{ESS}_{\mathrm{min}}}{t_{\mathrm{sampling}}}.$$

Using the minimum parameter-specific ESS provides a conservative summary (worst-case scenario) of each sampler's efficiency [1].

4.3 Results

4.3.1 Posterior Agreement

Table 4.2 summarizes the posterior means and standard deviations obtained for the two parameters across the three executions. The averaged posterior means ranged from 0.18509 to 0.19103 for μ and from 2334.5 to 2365.9 for ξ. The corresponding posterior standard deviations were also similar across the samplers, ranging from 0.08314 to 0.08379 for μ and from 944.4 to 956.0 for ξ. The variation across executions was small compared with the posterior uncertainty within each execution.



(a) RWMH (b) emcee

(c) PyMC Slice (d) PyMC SMC

(e) dynesty

Figure 4.1: Representative joint posterior distributions from the second execution for (a) RWMH, (b) emcee, (c) PyMC Slice, (d) PyMC SMC, and (e) dynesty.

4.3.2 Convergence Diagnostics

Table 4.3 summarizes the most conservative diagnostic values observed across the three executions. The bulk and tail ESS entries are the lowest values observed across all parameters and executions, while the R-hat entry is the highest observed value. The pipeline uses ESS greater than 400 and R-hat below 1.01 as thresholds for the formal chain-based assessment.

Table 4.3: Conservative diagnostic summaries across the three executions. Formal chain-based assessments are reported only for samplers producing independent Markov chains.

Sampler	Lowest bulk ESS	Lowest tail ESS	Highest R-hat	Chain-based assessment
RWMH	1198.9	1326.5	1.00346	PASS in 3/3
emcee	560.0	1928.8	1.02280	N/A
PyMC Slice	7439.8	7631.3	1.00090	PASS in 3/3
PyMC SMC	6607.8	6003.0	1.00059	N/A
dynesty	12723.1	12323.5	–	N/A

RWMH and PyMC Slice satisfied the chain-based ESS and R-hat thresholds in all three executions. For emcee, R-hat reached 1.02280 in one run. However, this is reported as a descriptive summary rather than a formal chain-based convergence decision.

Also for PyMC SMC, ESS diagnostic values describe the agreement for the final particle populations and, again, are not interpreted as conventional Markov chain diagnostics. Similarly, dynesty ESS values were obtained from its equal-weighted posterior representation and are not used as a conventional convergence assessment.

Dynesty terminated at the given stopping tolerance of $\Delta \log Z = 0.1$ in all three executions. Its estimated log-evidence was 1.03291 ± 0.01955 across the executions, while the average internal log-evidence uncertainty was 0.02081 ± 0.00027. Its Kish effective sample size was 8907.2 ± 40.7.

4.3.3 Computational Efficiency

Table 4.4 compares the number of surrogate-model evaluations, sampling time, and normalized ESS quantities across the three executions. ESS per evaluation and ESS per second were calculated separately for each execution before their means and standard deviations between executions were calculated.

Table 4.4: Computational summaries across the three executions, reported as mean ± SD.

Sampler	Evaluations	Time (s)	ESS/evaluation	ESS/s
RWMH	12025 ± 61	26.21 ± 0.32	0.10209 ± 0.00217	46.84 ± 0.58
emcee	32854 ± 318	72.60 ± 1.39	0.02043 ± 0.00328	9.26 ± 1.58
PyMC Slice	243995 ± 603	500.20 ± 4.70	0.03235 ± 0.00176	15.78 ± 0.93
PyMC SMC	84667 ± 10263	175.57 ± 21.22	0.08595 ± 0.00792	41.49 ± 4.52
dynesty	89220 ± 4975	193.15 ± 10.30	0.10004 ± 0.00548	46.20 ± 2.45

For dynesty, the efficiency metrics use Kish ESS rather than ArviZ ESS. RWMH required the fewest surrogate calls and had the shortest sampling time with its configuration settings. RWMH and dynesty produced similar ESS-based efficiency results. They are followed by PyMC SMC. PyMC Slice and emcee had the lowest efficiency values under the selected configurations.

4.4 Discussion

The results are consistent across the different samplers and runs. For this calibration problem, this indicates that the workflow is numerically stable, since the variation from one run to another is small compared with the uncertainty in the posterior itself. However, similar results from the samplers do not necessarily mean that the results are accurate. All of them use the same surrogate model, observation data, priors, and likelihood definition. Because of this, an error that is common to the model or implementation could appear in all samplers in a similar way.

The posterior distributions also show that the parameters are not equally well identified. The sampler-level average correlation between μ and ξ ranged from approximately 0.52 to 0.54 across the three runs. The posterior distribution of μ is noticeably narrower than its prior distribution. However, we cannot say the same for ξ as it remains broad and covers a large part of its prior range. The single impact-area observation might constrain combinations of the two parameters more strongly than it constrains ξ on its own. Different combinations of μ and ξ can produce similar agreement with the observed impact area [2].

The efficiency results in this study cannot support a general ranking for the samplers. For samplers, the stopping rules were not equivalent and the ESS-based efficiency metrics are not fully comparable for them. The observed differences only describe the selected configurations for this study.

4.4.1 Limitations

Each sampler was evaluated in three separate executions. This reduces dependence on a single result, but three executions are not enough to estimate reliably how much the results may vary from one execution to another. The random-number states were not recorded consistently for all samplers and executions, so the exact sample sequences cannot be reproduced in every case. This does not prevent the executions from being used to examine variation, but it limits exact computational reproducibility.

The exact software versions used during the benchmark are reported in Section 4.1. However, most dependencies are not fixed to exact versions in the Conda environment files. Recreating these environments later may therefore install different package versions. This also limits exact computational reproducibility, particularly because the serialized surrogate model may require a compatible Python and R software stack.

During sampling, every exception raised by a surrogate-model evaluation is currently handled by assigning a log-likelihood of negative infinity. This prevents an individual failed evaluation from stopping the sampler, but it does not distinguish an invalid parameter proposal from a communication or implementation error. The preserved logs do not show that this behavior affected the reported executions, but such failures could be hidden if they occur.

The same exception handling is used in all five sampler implementations:

```
try:
    prediction_mean = np.asarray(
        self.model([[*model_parameters]])
    )
except Exception:
    return -np.inf
```

The sampler settings were chosen as working configurations and were not optimized using a common criterion. Differences in computational performance may be related to both the sampling methods and their selected settings.

A further limitation is the lack of a single comparison metric that is equally suitable for all five methods. RWMH and PyMC Slice produce conventional Markov chains, emcee uses an ensemble of interacting walkers, PyMC SMC produces particle populations, and dynesty produces weighted nested-sampling points. Effective

sample size is calculated and interpreted differently for these outputs. Runtime and model-evaluation counts are more directly comparable, but they do not describe the quality of the posterior approximation on their own. The metrics used in this study provide a practical summary of the observed results, but they cannot support a fully equivalent comparison or a general ranking of the samplers.

The comparison did not include an independent reference posterior. From the perspective of philosophy of science, agreement among different methods can be viewed as converging evidence. When samplers based on different numerical approaches produce similar posterior estimates, this increases confidence that the results are not caused by a sampler-specific numerical error. In this study, this argument is limited to numerical consistency. Agreement among the samplers does not measure absolute numerical accuracy or establish that their shared model and statistical assumptions are correct. The study is also limited to one two-dimensional calibration problem, and the results may differ for higher-dimensional or more complex posterior distributions.

The likelihood uses the predictive mean of the RobustGaSP surrogate together with a fixed noise standard deviation. Surrogate uncertainty and an explicit model-discrepancy term are not included in the posterior. The resulting posterior distributions represent the common target used in the benchmark. They should not be interpreted as a complete uncertainty analysis of the physical Voellmy parameters [4, 31].

5 Summary and Future Work

This report presents `mcmc-bench`, a common, extensible framework for comparing sampling methods in Bayesian calibration problems. Five gradient-free sampler implementations were integrated and compared through a Nextflow workflow and a UM-Bridge interface to the RobustGaSP surrogate model.

5.1 Summary

The implementation in this study shows that sampler-specific environments can be integrated while the surrogate model, statistical problem, and output format are kept the same for Bayesian problem solving, which answers RQ1. For RQ2, it was assessed that measuring posterior accuracy requires an independent reference, which was not available for this study. Posterior agreement, convergence diagnostics, and efficiency still provide a practical comparison, provided that the diagnostics are interpreted per method. Chain-based statistics support a formal convergence decision only for RWMH and PyMC Slice, while the remaining samplers require their own termination and/or agreement arguments.

In this study, all of the samplers produced similar posterior means and standard deviations across three executions. The comparison did not identify a single best-performing sampler because the results depended on the selected efficiency metrics and ESS-based metrics are not fully equivalent for all samplers examined.

5.2 Future Work

The three executions help us to see the variation in the results and reduce the dependence on a single execution; however, more than three executions are needed to obtain a more reliable assessment of the variation in the results. Also, in future evaluations, higher-dimensional calibration problems should be added along with alternative models. Moreover, a reference posterior or a validated benchmark problem can be used for a better comparison and termination decision, yielding better

performance evaluations. That provides better options for sampler configuration settings and hence a fairer comparison. Also, the effects of the surrogate model uncertainty and model discrepancy should be examined.

For the `mcmc-bench` framework, further improvements should include computational reproducibility and automated validation. More detailed metadata files would make it possible to recreate the outputs more reliably if other users want to compare their results with this study. An example would be an optional seed setting, which would help the executions to be repeated exactly. Other effects on the results, such as the hardware and the communication state of the server, could then be compared. Moreover, automated tests can cover the likelihood calculation, sampler configurations, and output formats and they should run automatically when changes are pushed to the repository with Continuous Integration (CI).

Bibliography

1. J. Albert, C. Balázs, A. Fowlie, W. Handley, N. Hunt-Smith, R. Ruiz de Austri, and M. White. "A Comparison of Bayesian Sampling Algorithms for High-Dimensional Particle Physics and Cosmology Applications". *Computer Physics Communications* 315, 2025, p. 109756. DOI: 10.1016/j.cpc.2025.109756.
2. P. D. Arendt, D. W. Apley, and W. Chen. "Quantification of Model Uncertainty: Calibration, Model Discrepancy, and Identifiability". *Journal of Mechanical Design* 134:10, 2012, p. 100908. DOI: 10.1115/1.4007390.
3. M. Benning and M. Burger. "Modern Regularization Methods for Inverse Problems". *Acta Numerica* 27, 2018, pp. 1–111. DOI: 10.1017/S0962492918000016.
4. J. Brynjarsdóttir and A. O'Hagan. "Learning about Physical Parameters: The Importance of Model Discrepancy". *Inverse Problems* 30:11, 2014, p. 114007. DOI: 10.1088/0266-5611/30/11/114007.
5. J. Buchner. "UltraNest: A Robust, General Purpose Bayesian Inference Engine". *Journal of Open Source Software* 6:60, 2021, p. 3001. DOI: 10.21105/joss.03001.
6. P. Del Moral, A. Doucet, and A. Jasra. "Sequential Monte Carlo Samplers". *Journal of the Royal Statistical Society: Series B* (*Statistical Methodology*) 68:3, 2006, pp. 411–436. DOI: 10.1111/j.1467-9868.2006.00553.x.
7. P. Di Tommaso, M. Chatzou, E. W. Floden, P. Prieto Barja, E. Palumbo, and C. Notredame. "Nextflow Enables Reproducible Computational Workflows". *Nature Biotechnology* 35:4, 2017, pp. 316–319. DOI: 10.1038/nbt.3820.
8. S. Duane, A. D. Kennedy, B. J. Pendleton, and D. Roweth. "Hybrid Monte Carlo". *Physics Letters B* 195:2, 1987, pp. 216–222. DOI: 10.1016/0370-2693(87)91197-X.
9. dynesty Development Team. *dynesty API*. dynesty documentation. 2026. URL: https://dynesty.readthedocs.io/en/stable/api.html (visited on 08/15/2026).

10. D. Foreman-Mackey, D. W. Hogg, D. Lang, and J. Goodman. "emcee: The MCMC Hammer". *Publications of the Astronomical Society of the Pacific* 125:925, 2013, pp. 306–312. DOI: 10.1086/670067.

11. J. Goodman and J. Weare. "Ensemble Samplers with Affine Invariance". *Communications in Applied Mathematics and Computational Science* 5:1, 2010, pp. 65–80. DOI: 10.2140/camcos.2010.5.65.

12. M. Gu. "RobustCalibration: Robust Calibration of Computer Models in R". *The R Journal* 15:4, 2023, pp. 84–105. DOI: 10.32614/RJ-2023-085.

13. W. K. Hastings. "Monte Carlo Sampling Methods Using Markov Chains and Their Applications". *Biometrika* 57:1, 1970, pp. 97–109. DOI: 10.1093/biomet/57.1.97.

14. M. B. Heredia, N. Eckert, C. Prieur, and E. Thibert. "Bayesian Calibration of an Avalanche Model from Autocorrelated Measurements Along the Flow: Application to Velocities Extracted from Photogrammetric Images". *Journal of Glaciology* 66:257, 2020, pp. 373–385. DOI: 10.1017/jog.2020.11.

15. M. D. Hoffman and A. Gelman. "The No-U-Turn Sampler: Adaptively Setting Path Lengths in Hamiltonian Monte Carlo". *Journal of Machine Learning Research* 15, 2014, pp. 1593–1623.

16. M. C. Kennedy and A. O'Hagan. "Bayesian Calibration of Computer Models". *Journal of the Royal Statistical Society: Series B (Statistical Methodology)* 63:3, 2001, pp. 425–464. DOI: 10.1111/1467-9868.00294.

17. F. D. Moura Neto and A. J. da Silva Neto. *An Introduction to Inverse Problems with Applications*. 1st ed. Springer, Berlin, Heidelberg, 2013. ISBN: 978-3-642-32557-1. DOI: 10.1007/978-3-642-32557-1.

18. R. M. Neal. "Slice Sampling". *The Annals of Statistics* 31:3, 2003, pp. 705–767. DOI: 10.1214/aos/1056562461.

19. M. Pitkin, J. Midtbø, O. Abril, and R. Vieira. "Using a "Black Box" Likelihood Function". In: *PyMC Examples*. Ed. by PyMC Team. 2024. DOI: 10.5281/zenodo.5654871. URL: https://www.pymc.io/projects/examples/en/latest/howto/blackbox_external_likelihood_numpy.html.

20. PyMC Development Team. *pymc.smc.sample_smc*. PyMC documentation. 2026. URL: https://www.pymc.io/projects/docs/en/stable/api/generated/pymc.smc.sample_smc.html (visited on 07/18/2026).

21. PyMC Development Team. *pymc.step_methods.hmc.NUTS*. PyMC documentation. 2026. URL: https://www.pymc.io/projects/docs/en/stable/api/generated/pymc.step_methods.hmc.NUTS.html (visited on 07/18/2026).

22. PyMC Development Team. *pymc.step_methods.Slice*. PyMC documentation. 2026. URL: https://www.pymc.io/projects/docs/en/stable/api/generated/pymc.step_methods.Slice.html (visited on 07/18/2026).

23. R. van de Schoot, S. Depaoli, R. King, B. Kramer, K. Märtens, M. G. Tadesse, M. Vannucci, A. Gelman, D. Veen, J. Willemsen, and C. Yau. "Bayesian Statistics and Modelling". *Nature Reviews Methods Primers* 1:1, 2021, pp. 1–26. DOI: 10.1038/s43586-020-00001-2.

24. L. Seelinger, V. Cheng-Seelinger, A. Davis, M. Parno, and A. Reinarz. "UM-Bridge: Uncertainty Quantification and Modeling Bridge". *Journal of Open Source Software* 8:83, 2023, p. 4748. DOI: 10.21105/joss.04748.

25. J. Skilling. "Nested Sampling for General Bayesian Computation". *Bayesian Analysis* 1:4, 2006, pp. 833–859. DOI: 10.1214/06-BA127.

26. J. S. Speagle. "dynesty: A Dynamic Nested Sampling Package for Estimating Bayesian Posteriors and Evidences". *Monthly Notices of the Royal Astronomical Society* 493:3, 2020, pp. 3132–3158. DOI: 10.1093/mnras/staa278.

27. A. M. Stuart. "Inverse Problems: A Bayesian Perspective". *Acta Numerica* 19, 2010, pp. 451–559. DOI: 10.1017/S0962492910000061.

28. A. Vehtari, A. Gelman, D. Simpson, B. Carpenter, and P.-C. Bürkner. "Rank-Normalization, Folding, and Localization: An Improved $\widehat{R}$ for Assessing Convergence of MCMC". *Bayesian Analysis* 16:2, 2021, pp. 667–718. DOI: 10.1214/20-BA1221.

29. M. J. Williams, J. Veitch, and C. Messenger. "Nested Sampling with Normalizing Flows for Gravitational-Wave Inference". *Physical Review D* 103:10, 2021, p. 103006. DOI: 10.1103/PhysRevD.103.103006.

30. K. W. K. Wong, M. Gabrié, and D. Foreman-Mackey. "flowMC: Normalizing Flow Enhanced Sampling Package for Probabilistic Inference in JAX". *Journal of Open Source Software* 8:83, 2023, p. 5021. DOI: 10.21105/joss.05021.

31. A. Yildiz, H. Zhao, and J. Kowalski. "Computationally-Feasible Uncertainty Quantification in Model-Based Landslide Risk Assessment". *Frontiers in Earth Science* 10, 2023, p. 1032438. DOI: 10.3389/feart.2022.1032438.

32. H. Zhao and J. Kowalski. "Bayesian Active Learning for Parameter Calibration of Landslide Run-Out Models". *Landslides* 19:8, 2022, pp. 2033–2045. DOI: 10.1007/s10346-022-01857-z.